

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра САПР

Курсовой проект
по дисциплине
“Структуры и алгоритмы компьютерной обработки данных”.

Тема: «Разработка комплекса программ реализации алгоритмов работы с
АВЛ-деревьями»

Направление подготовки – 02.03.03 Математическое обеспечение и
администрирование информационных систем
Профиль подготовки – Администрирование информационных систем

Выполнил студент : _____ Петров А.А.
Группа: _____ 18BA1
Руководитель :
к.т.н., доцент _____ Глотова Т.В.

Здесь лист ТЗ

РЕФЕРАТ

Пояснительная записка содержит 40 страниц, 8 рисунков, 3 источника, 2 приложения.

ПРОГРАММНЫЙ КОМПЛЕКС, C++, АБСТРАКТНЫЙ ТИП ДАННЫХ, АВЛ-ДЕРЕВО, АЛГОРИТМ БАЛАНСИРОВКИ АВЛ-ДЕРЕВА, АЛГОРИТМ ПОИСКА.

Объектом разработки является программный комплекс реализации алгоритмов работы с АВЛ-деревом.

Целью курсового проекта является разработка программного комплекса реализации алгоритмов добавления, удаления, поиска в АВЛ дереве и исследование их эффективности.

Разработана структура программного комплекса, АД - класс абстрактного типа данных АВЛ дерево., проведены исследования эффективности алгоритмов.

Программный комплекс реализован на языке C++ в среде разработки MS Visual Studio 2010.

В приложении приведены структурная схема, схемы алгоритмов и исходный код программы.

При разработке программного комплекса была использована технология объектно-ориентированного программирования.

					<i>КП. 020303. 03.20.81 01</i>		
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>			
<i>Разраб.</i>					<i>Реферат</i> <i>Пояснительная записка</i>		
<i>Провер.</i>	<i>Глотова Т.В.</i>						
<i>Реценз.</i>							
<i>Н. Контр.</i>							
<i>Утверд.</i>							
					<i>Лит.</i>	<i>Лист</i>	<i>Листов</i>
						1	1
					<i>ФВТ группа 18ВА1</i>		

Оглавление

.....	3
1. Анализ ТЗ.	5
2. Постановка задачи.	6
3. Описание структуры данных.	7
4. Разработка детальных алгоритмов решения.	9
5. Разработка структуры программы.	11
6. Разработка программы.	12
7. Описание программы.	13
8. Экспериментальная часть.	16
9. Заключение.	19
Список литературы	20
ПРИЛОЖЕНИЕ А. Графическая часть.	21
ПРИЛОЖЕНИЕ Б. Исходный код программы.	

1. Анализ ТЗ.

В процессе выполнения курсового проекта должны быть разработаны абстрактный тип данных (АТД) АВЛ-дерево, структура программного комплекса, детальные алгоритмы работы с АВЛ-деревьями: поиска в дереве, добавления и удаления вершин с балансировкой дерева. Должен быть выполнен сравнительный анализ алгоритмов, приведено исследование эффективности работы алгоритмов. Работа алгоритмов должна быть представлена скриншотами работы программы на контрольных примерах. Программный комплекс должен быть разработан на языке программирования C++ в среде разработки VisualStudio2010.

2. Постановка задачи.

Тема курсового проекта: «Разработка комплекса программ реализации алгоритмов работы с АВЛ-деревьями».

Цель работы: разработать программу создания АВЛ-дерева: добавление, удаление, поиск элементов, обход дерева с выводом на печать.

Области применения деревьев: для хранения информации, как таковой; в процедурах поиска и сортировки; для описания хранения файлов на дисковых носителях; при построении эффективных кодов для сжатия информации, диспетчер динамической памяти, основанный на АВЛ-деревьях[1,2].

АВЛ-дерево (AVL tree) – сбалансированное по высоте двоичное дерево поиска, в котором у любой вершины высота левого и правого поддеревьев различаются не более чем на 1[1-3].

Основная идея Если вставка или удаление элемента приводит к нарушению сбалансированности дерева, то выполняется его балансировка Коэффициент сбалансированности узла (balance) – это разность высот его левого и правого поддеревьев В АВЛ-дереве коэффициент сбалансированности любого узла принимает значения из множества $\{-1, 0, 1\}$ Высота узла – это длина наибольшего пути от него до дочернего узла, являющего листом Высота листа равна 0. Высота пустого поддерева (NULL) равна -1.

После добавления нового элемента необходимо обновить коэффициенты сбалансированности родительских узлов.

Если любой родительский узел принял значение -2 или 2, то необходимо выполнить балансировку поддерева путем поворота (rotation).

Типы поворотов:

- Одиночный правый поворот
- Одиночный левый поворот
- Двойной лево-правый поворот
- Двойной право-левый поворот[3]

3. Описание структуры данных.

AVLTREE – AVL-дерево

Данные

- int key; - ключ
- const char* - произвольные данные
- intbalance; Коэффициент сбалансированности узла– это разность высот его левого и правого поддеревьев

В AVL-дереве коэффициент сбалансированности любого узла принимает значения из множества $\{-1, 0, 1\}$

Операции

— Конструктор(key)

Вход: key.

Начальные значения: данные для инициализации.

Процесс: инициализация данных.

Постусловие: состояние структуры и параметров после инициализации.

— Lookup(T, key)

Вход: key, дерево T.

Предусловия: создан объект класса

Процесс: поиск заданного узла.

Выход: узел, который был найден.

Постусловие: Нет.

— Remove(T, key, avltree*)

Вход: key, дерево T, узел.

Предусловия: создан объект класса.

Процесс: поиск и удаление узла.

Выход: изменение баланса.

Постусловие: в АД уменьшилось количество узлов, изменился баланс дерева.

— **Add(T, key)**

Вход: key, дерево T.

Предусловия: создан объект класса

Процесс: добавление узла.

Выход: изменение баланса.

Постусловие: в АДТ увеличилось количество узлов, изменился баланс дерева.

4. Разработка детальных алгоритмов решения.

4.1. Алгоритм

поиска

Сравнивая ключ, заданный пользователем, с ключом в узле, алгоритм двигается от корня либо вправо, либо влево, в зависимости от того меньше заданный ключ или больше чем в узле.

Когда алгоритм находит заданный ключ, то возвращает указатель на этот узел, если же ключа найти не удалось, выводит сообщение о неудачном поиске.

4.2. Алгоритм

добавления

узла

(рекурсивный)

Если элемент с добавляемым ключом уже существует, то просто возвращает 0. Если же нет, то сохраняется текущий баланс начинается рекурсивный алгоритм, который сравнивает ключ с текущим узлом, алгоритм двигается от корня либо вправо, либо влево, в зависимости от того меньше заданный ключ или больше чем в узле.

Для конкретики возьмем правую ветку. Если правый потомок не $=0$, то баланс текущего узла увеличивается на возвращаемое значение от вызова алгоритма добавления узла правого потомка, после вызывается баланс дерева, который выполняет нужный поворот, если он требуется.

Если же правый потомок равен нулю, то создается новый узел-потомок, и увеличивается баланс текущего узла на 1. Аналогично и с левой веткой, только баланс уменьшается в обоих случаях. В конце алгоритма проверяется: если прошлый баланс был $=0$, то если настоящий баланс не равен прошлому, то возвращается 1, если был равен -0 , а если прошлый баланс не был равен 0, то всегда возвращается 0.

4.3. Алгоритм

удаления

узла

(рекурсивный)

Сохраняется текущий баланс. Если заданный ключ равен ключу в узле, то если у текущего узла нет потомков, тогда если это правый потомок предыдущего узла, который вызывал алгоритм, то удаляется правый потомок, в ином случае – левый и возвращается 1. Если же заданный ключ

не равен текущему ключу, то если баланс текущего узла равен или больше 0, то если правый потомок не равен 0, создается указатель на правый потомок. Идем до последнего левого потомка от правого. Меняем местами текущий элемент и до которого дошли. Баланс текущего элемента уменьшаем на возвращаемое значение от вызова удаления правого потомка. Если баланс текущего узла меньше 0, то выполняем аналогичные действия для левого потомка с инверсией. Если же ключ не равен текущему, то если он больше и правый потомок не равен 0, то баланс текущего элемента уменьшаем на возвращаемое значение от вызова удаления правого потомка. Потом вызываем, баланс дерева, который выполняет нужный поворот, если он требуется. Если правый потомок = 0, то узел не найден. Аналогично для левого потомка выполняем те же действия с инверсией, если ключ меньше текущего. . В конце алгоритма проверяется: если прошлый баланс был =0, то если настоящий баланс не равен прошлому, то возвращается 1, если был равен – 0, а если прошлый баланс не был равен 0, то всегда возвращается 0.

5. Разработка структуры программы.

Разработанная программный комплекс состоит из следующих элементов :

avltree – класс АВЛ-дерево

doubletime – вспомогательная функция подсчета времени

Класс содержит следующие приватных свойства и методы:

intkey – хранит ключ узла

intbalance- хранит баланс узла

constchar * value – хранит произвольное значение

avltree *left – хранит указатель на левого потомка узла

avltree *right – хранит указатель на правого потомка узла

voidavltree::small_r– осуществляет малый правый поворот

oidavltree::big_r– осуществляет большой правый поворот

voidavltree::small_l- осуществляет малый левый поворот

voidavltree::big_l- осуществляет большой левый поворот

oidavltree:: TurnAround–выполняет проверку на баланс и на основе этого выбирает какой из поворотов сделать

voidavltree::printavl – выводит дерево на печать

А также из открытых методов:

avltree::avltree – создает объект класса avltree

voidavltree::print– вспомогательная функция печати, которая передает указатель на корень

intavltree:: remove – удаляет элемент

avltree* avltree:: lookup –ищет заданный элемент

intavltree::add-добавляет элемент

6. Разработка программы.

Класс АВЛ дерева avltree разработан на языке C++:

```
class avltree //класс АВЛ-дерево
```

```
{
```

Структура данных АВЛ дерева:

```
private: //приватные поля
```

```
    int key; //ключ
```

```
    int balance; //баланс
```

```
    const char * value; //значение
```

```
    avltree *left; //левый потомок
```

```
    avltree *right; //правый потомок
```

Методы реализации:

```
    void small_r(); //малый правый поворот
```

```
    void big_r(); //большой правый поворот
```

```
    void small_l(); //малый левый поворот
```

```
    void big_l(); //большой левый поворот
```

```
    void TurnAround(); //выполняет проверку на баланс и на основе этого
```

выбирает какой из поворотов сделать

```
    void printavl(avltree*, int); // выводит дерево на печать, требует указатель на
```

узел и число отступов для печати

Интерфейс класса:

```
public:
```

```
    avltree(int, const char * value); // создает объект класса avltree
```

```
    void print(int l); // вспомогательная функция печати
```

```
    int remove(int, avltree*); // удаляет элемент, требует ключ и указатель на  
родителя (для корня по умолчанию=0)
```

```
    avltree* lookup(int); //ищет заданный элемент по введенному ключу
```

```
    int add(int, const char * value); //добавляет элемент по ключу и значению,  
заданными пользователем
```

```
};
```

Замечание Должно быть представлены подробные описания

Схемы алгоритмов поиска, удаления, добавления и печати класса представлены в Приложении А.

7. Описание программы.

При запуске программа позволяет создать дерево из заданного числа узлов, добавить, удалить его или найти узел по ключу, а также напечатать дерево.

На вход программе подаются десять значений в заданном порядке: 1, 41, 67, 34, 0, 69, 24, 78, 58, 62. Должно быть построено дерево, показанное на рисунке 1.

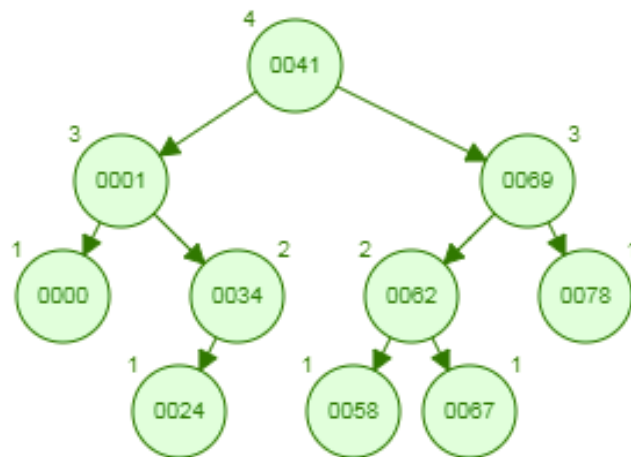


Рисунок 1

Результат работы программы показан на рисунке 2. Видим, что программа строит дерево правильно.

```
C:\Users\Grigoriy\source\repos\СИА курсовая\Debug\СИА курсовая.exe
Enter a number of nodes
Enter -1 if tou want to back: 10
41 67 34 0 69 24 78 58 62 Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter: 4

78
69
67
62
58
41
34
24
1
0
Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter:
```

Рисунок 2

Удалим, например, корень дерева – 41. Мы должны будем получить такое дерево после удаления как на рисунке 3.

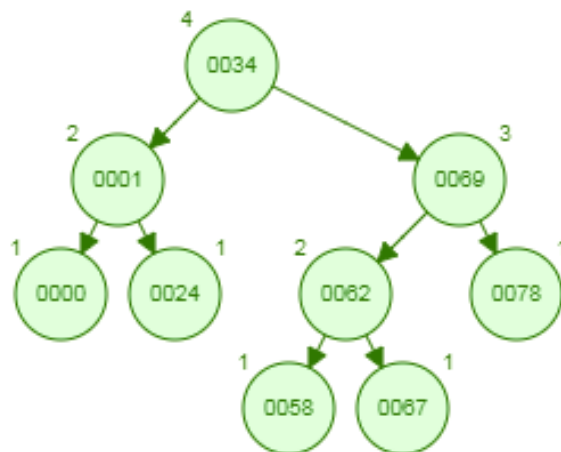


Рисунок 3

Результат показан на рисунке 4. Все выполнено верно.

```

C:\Users\Grigoriy\source\repos\СИА курсовая\Debug\СИА курсовая.exe
Enter: 2
You gonna dell the node
Enter key: 41
Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter: 4
    78
  69
    67
    62
    58
34
  24
  1
  0
Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter:

```

Рисунок 4

Добавим узел – 59, должно получиться дерево как на рисунке 5.

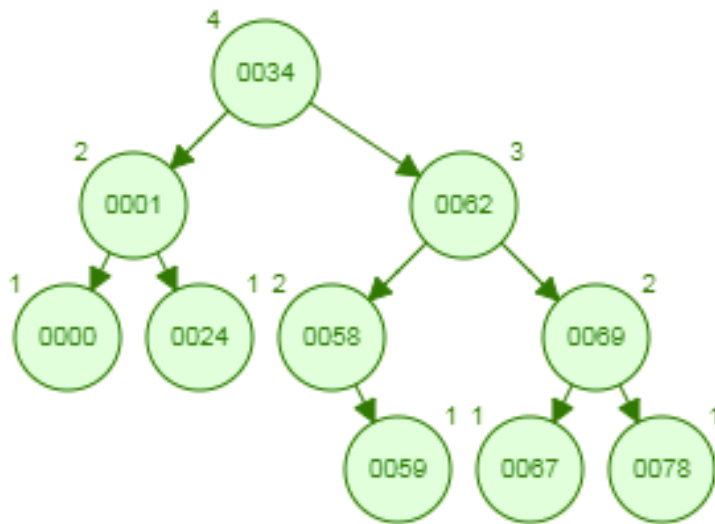


Рисунок 5

Результат работы программы показан на рисунке 6. Все выполнено верно.

```

C:\Users\Grigoriy\source\repos\СИА курсовая\Debug\СИА курсовая.exe
You gonna add the node
Enter key: 59
Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter: 4

  78
 69
 67
 62
 59
 58
34
 24
 1
 0
Enter number 1 if you want to add node;
Enter number 2 if tou want to dell node;
Enter number 3 if you want to find node
Enter number 4 if tou want to print avltree
Enter number 5 if tou want to exit
Enter:
  
```

Рисунок 6

При тестировании программа проверена на различных данных и показала свою успешную работоспособность.

8. Экспериментальная часть.

Исследования проводились на различном количестве данных:143000, 286000, 429000, 572000, 715000, 858000, 1001000. Основными критериями оценки были трудоёмкость(операции) и время (секунды).

Время, с			
Количество элементов	Поиск	Добавление	Удаление
0	0	0	0
143000	0,468	7,846	2,512
286000	0,899	15,99	5,288
429000	1,373	24,555	8,143
572000	1,807	32,807	11,17
715000	2,324	41,05	14,196
858000	2,871	49,665	17,207
1001000	3,26	58,251	20,374

Таблица 1. Время работы алгоритмов

Трудоёмкость, операции			
Количество элементов	Поиск	Добавление	Удаление
0	0	0	0
143	1704	10100	7780
286	3889	22932	17565
429	5992	36803	27709
572	9136	51192	39000

715	11424	66350	50376
858	13712	81508	61318
1001	16000	96666	72763

Таблица 2. Практическая трудоемкость алгоритмов

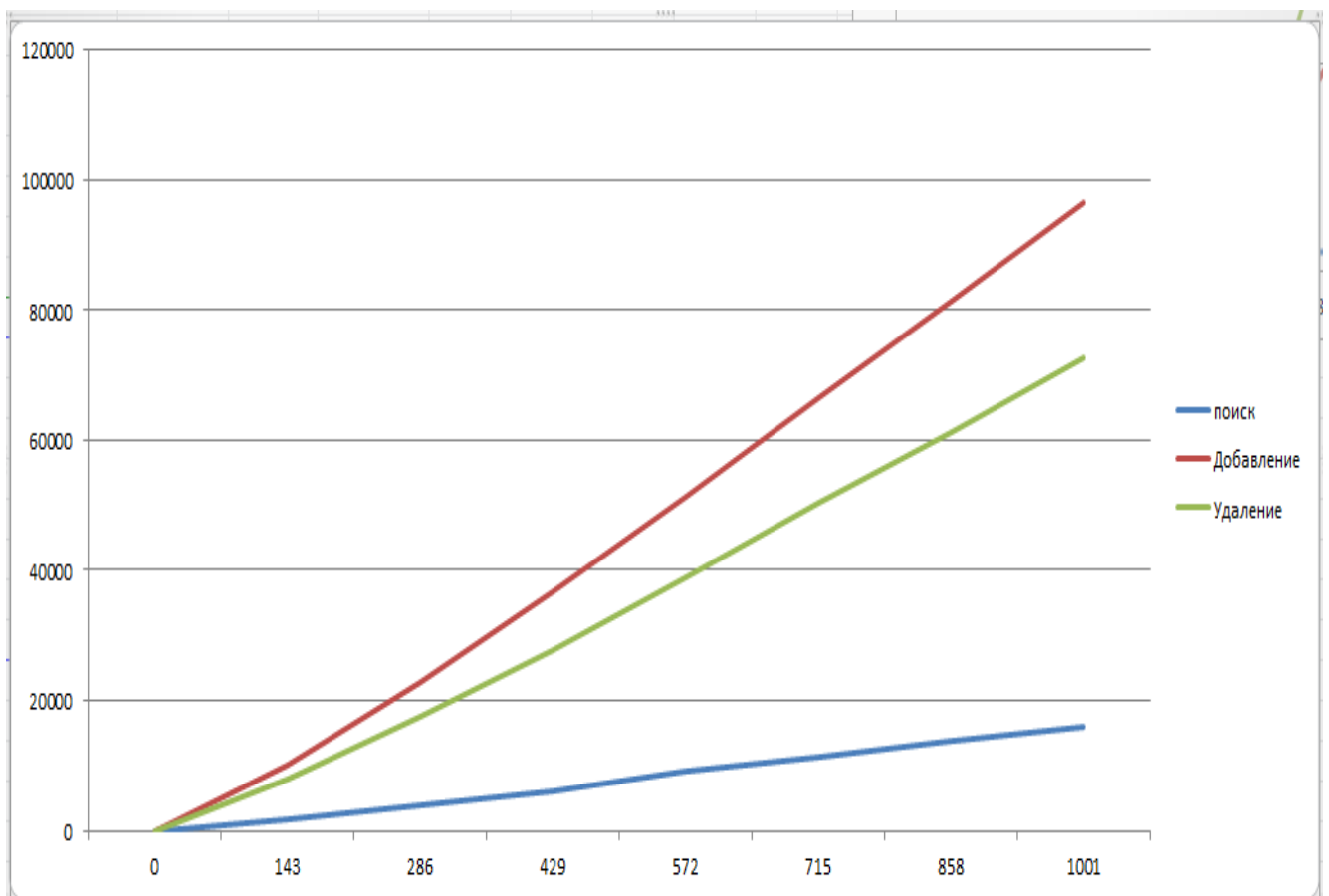


Рисунок 7. График зависимости трудоемкости от количества элементов

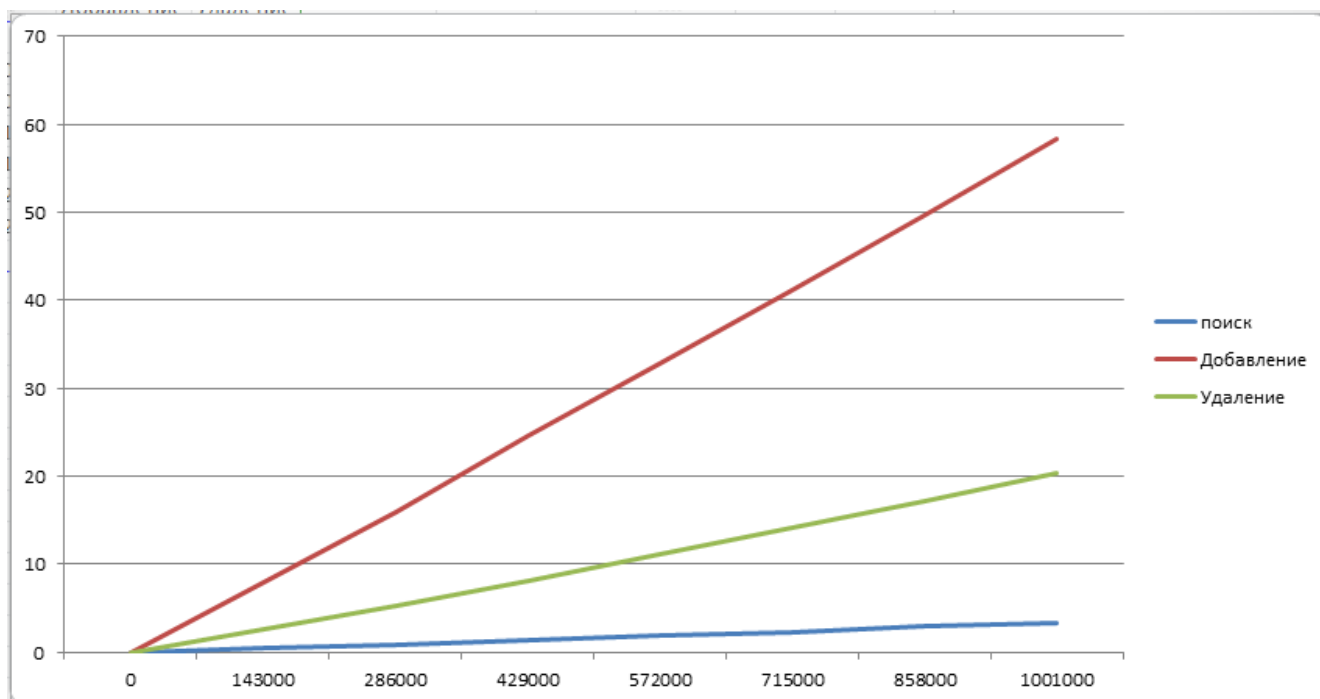


Рисунок 8. График зависимости времени работы алгоритмов от количества элементов

Сравнивая результаты экспериментальных исследований, можно увидеть, что алгоритм поиска выполняется за меньший промежуток времени, чем алгоритмы добавления и удаления.

Результаты практических измерений подтвердили теоретические оценки эффективности алгоритмов поиска, добавления и удаления – все $O(\log N)$.

Замечание указать параметра компьютера, на котором производились исследования

9. Заключение.

В процессе курсового проектирования были разработаны абстрактный тип данных АВЛ-дерева `avltree`, структура программного комплекса, реализовано алгоритмы: поиска, добавления, удаления с балансировкой АВЛ-дерева,. В пояснительной записке приведен график исследования времени работы и трудоемкости алгоритмов, схемы алгоритмов, структурная схема программы, описание АТД, скриншоты. Комплекс программ разработан на языке программирования C++ в среде VisualStudio 2010.

Программа реализует следующие функции:

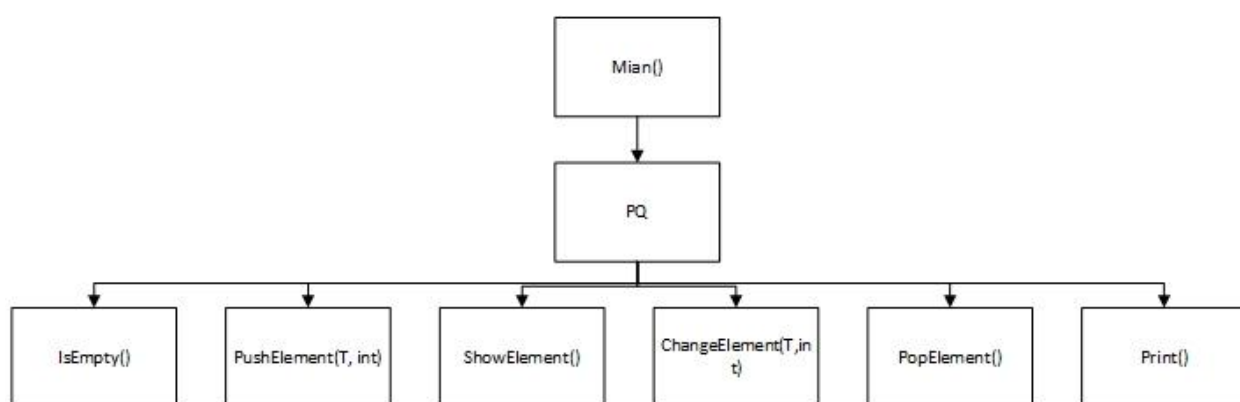
- Формирования АВЛ-дерева;
- Вывод его на печать;
- Добавление узла;
- Поиск узла;
- Удаление узла;

Требование технического задания выполнено в полном объеме.

Список литературы

1. Роберт Седжвик - Фундаментальные алгоритмы C++. Части 1-4, 2001. -687с
2. Вирт Н. Алгоритмы и структуры данных. – М.: Мир, 1989. – 360 с.
3. Лекция 3: АВЛ-деревья (AVL tree)[Электронный ресурс]. –
URL :<http://www.mkurnosov.net/teaching/uploads/DSA/dsa-fall-lecture3.pdf>
(Дата доступа : 22.03.2018).

ПРИЛОЖЕНИЕ А. Графическая часть.



					КП. 020303. 03.20 81 01			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.					Структурная схема		Лист	Листов
Провер.	Глотова Т.В.						1	1
Реценз.							ФВТ группа 16ВА1	
Н. Контр.								
Утверд.								
					Пояснительная записка			

